NAMA : I MADE MERTA RAIYANA

NIM : 240040014

KELAS : CA244

TUGAS 1

APLIKASI: INVENTARIS KATALOG UMKM

Deskripsi Singkat:

Aplikasi berbasis platform menggunakan Flutter untuk membantu pelaku UMKM kerajinan di Gianyar dalam mendokumentasikan, melacak inventaris, serta menampilkan katalog produk lokal secara terstruktur menggunakan penyimpanan database SQLite lokal dan arsitektur MVVM.

KEBUTUHAN FUNGSIONAL:

ID	Kebutuhan Fungsional	Prioritas	Komponen / Package
FR-01	Sistem dapat menampilkan daftar katalog produk UMKM secara rapi.	Must	ListView.builder + Card
FR-02	Pengguna dapat melihat detail informasi produk (harga, stok, deskripsi).	Must	Column + Text
FR-03	Admin dapat menambahkan data produk baru ke dalam sistem katalog.	Must	Form + TextFormField
FR-04	Admin dapat mengubah/memperbarui data produk yang sudah ada.	Must	Form + TextFormField
FR-05	Admin dapat menghapus data produk dari sistem dengan konfirmasi pembatalan.	Must	AlertDialog (Batal / Hapus)

KEBUTUHAN NON-FUNGSIONAL:

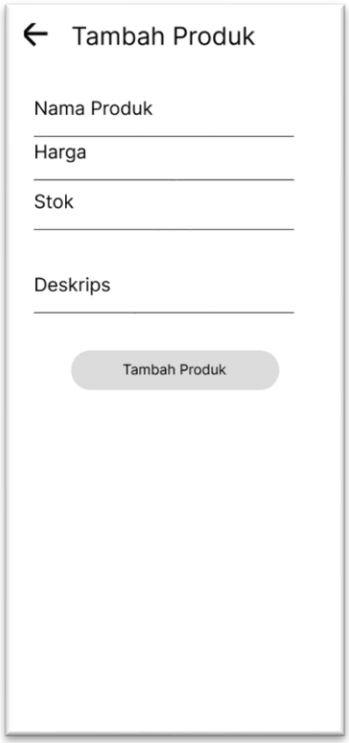
- **NFR-01 (Kompatibilitas):** Sistem dapat dijalankan dengan lancar di perangkat ponsel cerdas Android melalui emulator maupun perangkat fisik.
- **NFR-02 (Maintainability):** Kode program dipisah secara terstruktur per lapisan (*separation of concerns*) menggunakan pola arsitektur View, ViewModel, dan Repository untuk mempermudah perawatan kode ke depan.
- **NFR-03 (Reliabilitas/Persistensi):** Data katalog yang diinput oleh pengguna harus tetap tersimpan secara aman dan permanen di dalam penyimpanan lokal perangkat menggunakan SQLite, meskipun aplikasi dimatikan secara paksa.

RANCANGAN UI (WIREFRAME):

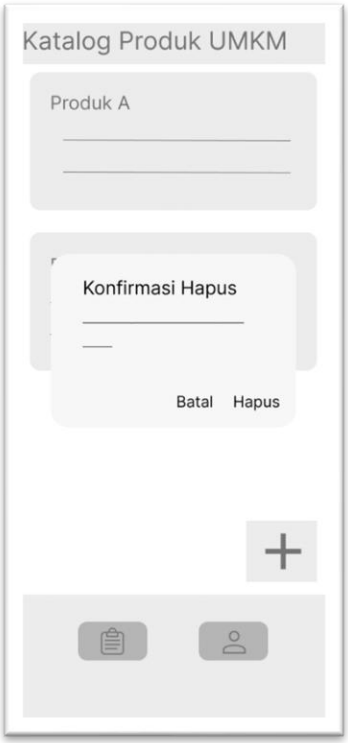












PRINSIP UI/UX YANG DITERAPKAN:

1. Hierarki Visual (*Visual Hierarchy*)

- **Penerapan pada Wireframe:**
    - Pada **Halaman Katalog**, nama produk seperti "Produk A" dan "Produk B" diletakkan di bagian atas kartu (Card) dengan ukuran teks yang lebih besar dan tebal dibandingkan informasi harga atau stok di bawahnya.
    - Tombol Tambah Data (+) menggunakan komponen *Floating Action Button* (FAB) yang diletakkan secara melayang di sudut kanan bawah untuk menarik perhatian instan admin.
- **Alasan UX:** Memandu mata Admin agar langsung mengenali informasi utama (Nama Produk) sebelum membaca detail sekunder, serta mempermudah eksekusi aksi utama (menambah data).

2. Konsistensi (*Consistency & Standards*)

- **Penerapan pada Wireframe:**
    - Struktur navigasi bawah (NavigationBar) tetap konsisten berada di posisi yang sama pada **Halaman Katalog** dan **Halaman Profil**.
    - Komponen input form pada halaman **Tambah Produk** dan **Ubah Produk** menggunakan struktur garis placeholder horizontal bertingkat yang seragam.
- **Alasan UX:** Memanfaatkan memori jangka pendek pengguna. Ketika Admin sudah memahami cara berinteraksi di satu halaman, mereka tidak perlu belajar ulang saat berpindah ke halaman lain karena polanya tetap sama.

3. Jangkauan Jempol & Target Sentuh (*Thumb Zone & Touch Targets*)

- **Penerapan pada Wireframe:**
    - Navigasi utama (NavigationBar) serta tombol krusial seperti aksi Tambah Data (+) dan tombol Keluar seluruhnya ditempatkan di area bawah kanvas layar ponsel.
- **Alasan UX:** Memenuhi aturan ergonomi *Material Design 3*. Penempatan elemen penting di area bawah (zona jempol) memastikan Admin dapat mengoperasikan aplikasi dengan satu tangan secara nyaman tanpa harus meregangkan jari ke area atas layar.

4. Pencegahan Kesalahan & Umpan Balik (*Error Prevention & Feedback*)

- **Penerapan pada Wireframe:**
    - Pada komponen **Konfirmasi Hapus**, aplikasi memunculkan sebuah kotak dialog pop-up (AlertDialog) dengan dua opsi aksi yang jelas: Batal dan Hapus.
- **Alasan UX:** Menerapkan prinsip *Forgiving UI* (antarmuka yang pemaaf). Sistem mencegah hilangnya data secara tidak sengaja apabila Admin salah menekan ikon tempat sampah, memberikan kesempatan bagi pengguna untuk membatalkan aksinya.

5. Kesederhanaan & Minimalis (*Simplicity / Aesthetic & Minimalist Design*)

- **Penerapan pada Wireframe:**
    - Halaman **Form Login** dan **Profil** dirancang sangat sederhana. Hanya terdapat komponen input esensial (Email, Password, tombol Masuk) dengan sedikit elemen dekoratif, seperti warna background.
- **Alasan UX:** Meminimalkan beban kognitif pengguna (*Cognitive Load*). Admin dapat fokus menyelesaikan satu tugas utama per layar (misal: mengisi form atau melihat data profil) dengan tingkat efisiensi maksimal.

RANCANGAN DATA:

Aplikasi Inventaris Katalog Digital Produk UMKM ini mengimplementasikan database lokal SQLite melalui package sqflite. Berikut adalah struktur rancangan tabel data utama yang digunakan untuk menyimpan entitas objek produk:

A. Sintaks Pembuatan Tabel (CREATE TABLE)

```
CREATE TABLE products (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL,
 price INTEGER NOT NULL,
 stock INTEGER NOT NULL,
 description TEXT NOT NULL
);
```

B. Atribut Entitas Data

Entitas data utama memiliki 5 atribut (1 identitas otomatis dan 4 atribut data) sebagai berikut:

- id: Identitas unik setiap produk berjenis data INTEGER yang bertambah otomatis (*Autoincrement*).
- name: Nama produk kerajinan berjenis data teks (TEXT).
- price: Harga jual produk dalam satuan Rupiah berjenis data angka bulat (INTEGER).
- stock: Jumlah ketersediaan barang di gudang berjenis data angka bulat (INTEGER).
- description: Detail deskripsi atau kategori produk berjenis data teks (TEXT).

C. Alasan Memilih SQLite Dibanding SharedPreferences

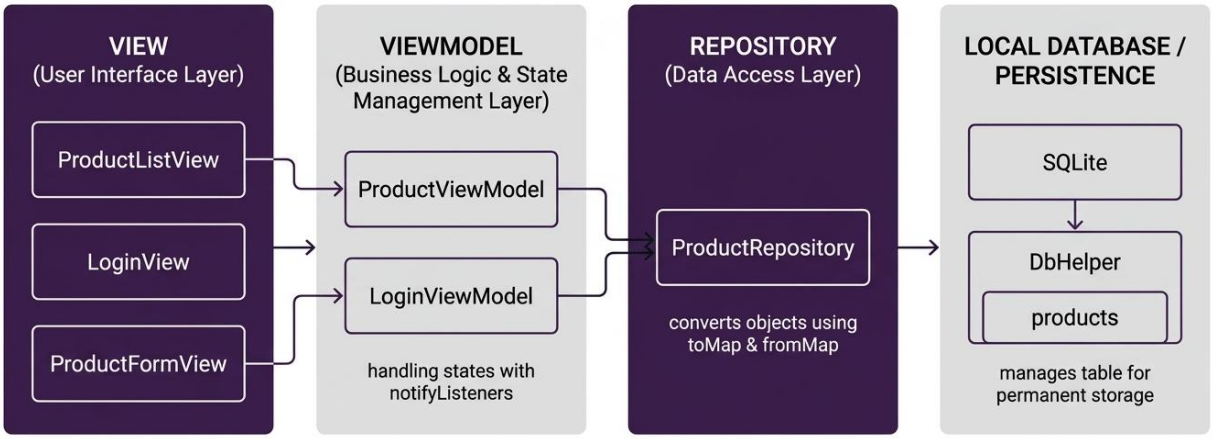
Dalam pengembangan aplikasi berbasis Flutter ini, pemilihan **SQLite** sebagai media penyimpanan utama didasarkan pada pertimbangan analisis teknis berikut:

Aspek Perbandingan	SQLite (Dipilih)	SharedPreferences (Dieliminasi)
Karakteristik Data	Cocok untuk data relasional, kompleks, dan berstruktur tabel.	Hanya cocok untuk data sederhana berpasangan *Key-Value*.
Representasi Objek	Mampu menyimpan objek model utuh dengan banyak atribut secara efisien.	Harus dikonversi manual ke string JSON jika objeknya kompleks.
Skalabilitas Data	Sangat mumpuni untuk menampung ratusan hingga ribuan baris data katalog.	Performa akan menurun drastis jika digunakan untuk data massal.
Kemampuan Query	Mendukung pencarian, pengurutan, dan manipulasi data tingkat lanjut via SQL.	Tidak memiliki kapabilitas query data dinamis.

Kesimpulan: Karena entitas utama aplikasi adalah "Produk" yang memiliki banyak atribut terikat (Nama, Harga, Stok, Deskripsi) dan jumlah datanya dinamis (bisa terus bertambah), maka penggunaan database relasional lokal **SQLite jauh lebih tepat dan andal** dibandingkan *SharedPreferences* yang hanya ideal untuk menyimpan data konfigurasi sederhana (seperti status *Dark Mode* atau token login singkat).

ALUR ARSITEKTUR CRUD (MVVM + REPOSITORY):

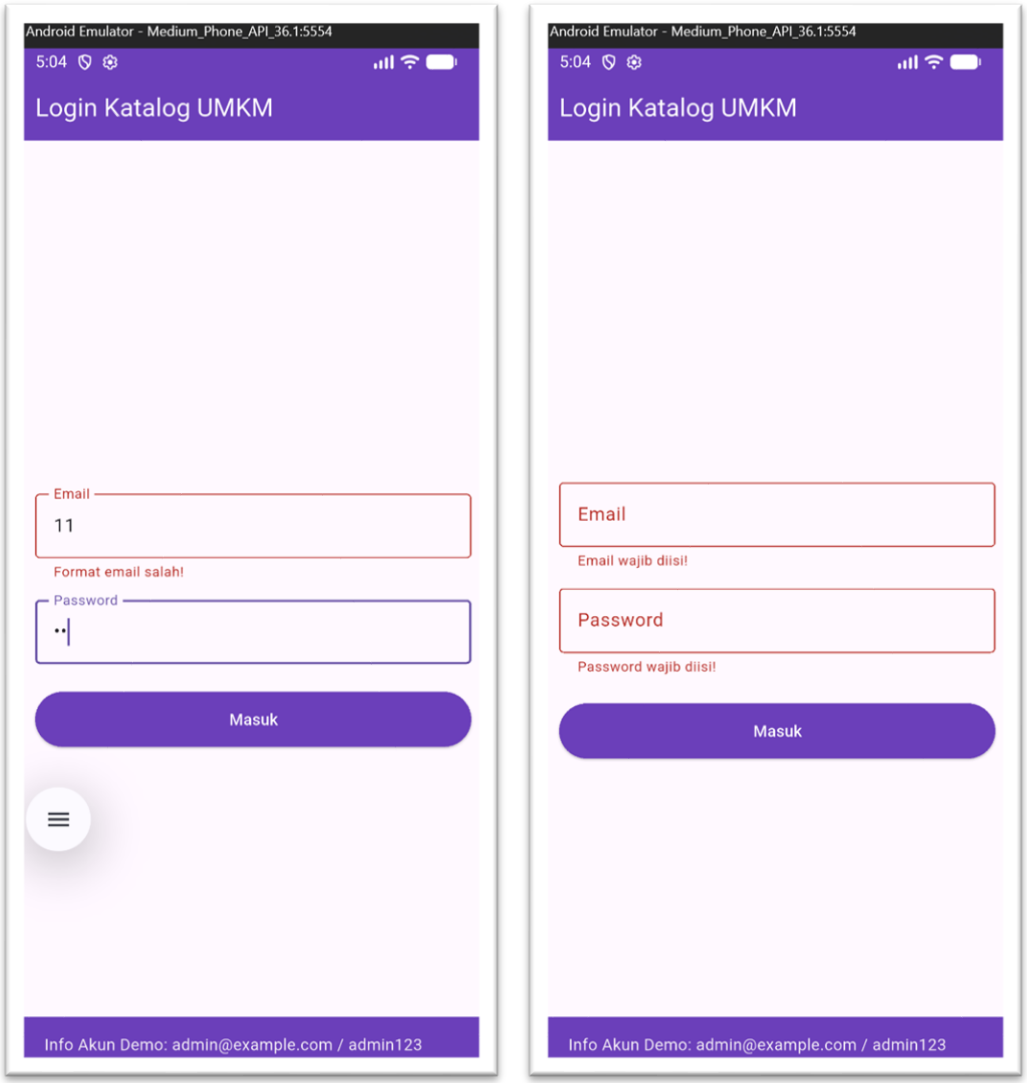
Aplikasi ini menerapkan pola arsitektur MVVM (View - ViewModel - Repository) untuk memisahkan secara tegas antara logika bisnis dengan tampilan antarmuka.



Berikut adalah penjelasan langkah demi langkah aliran data dari aksi pengguna di layar hingga tersimpan secara permanen di database lokal (Contoh Alur: **Operasi Create / Tambah Data**):

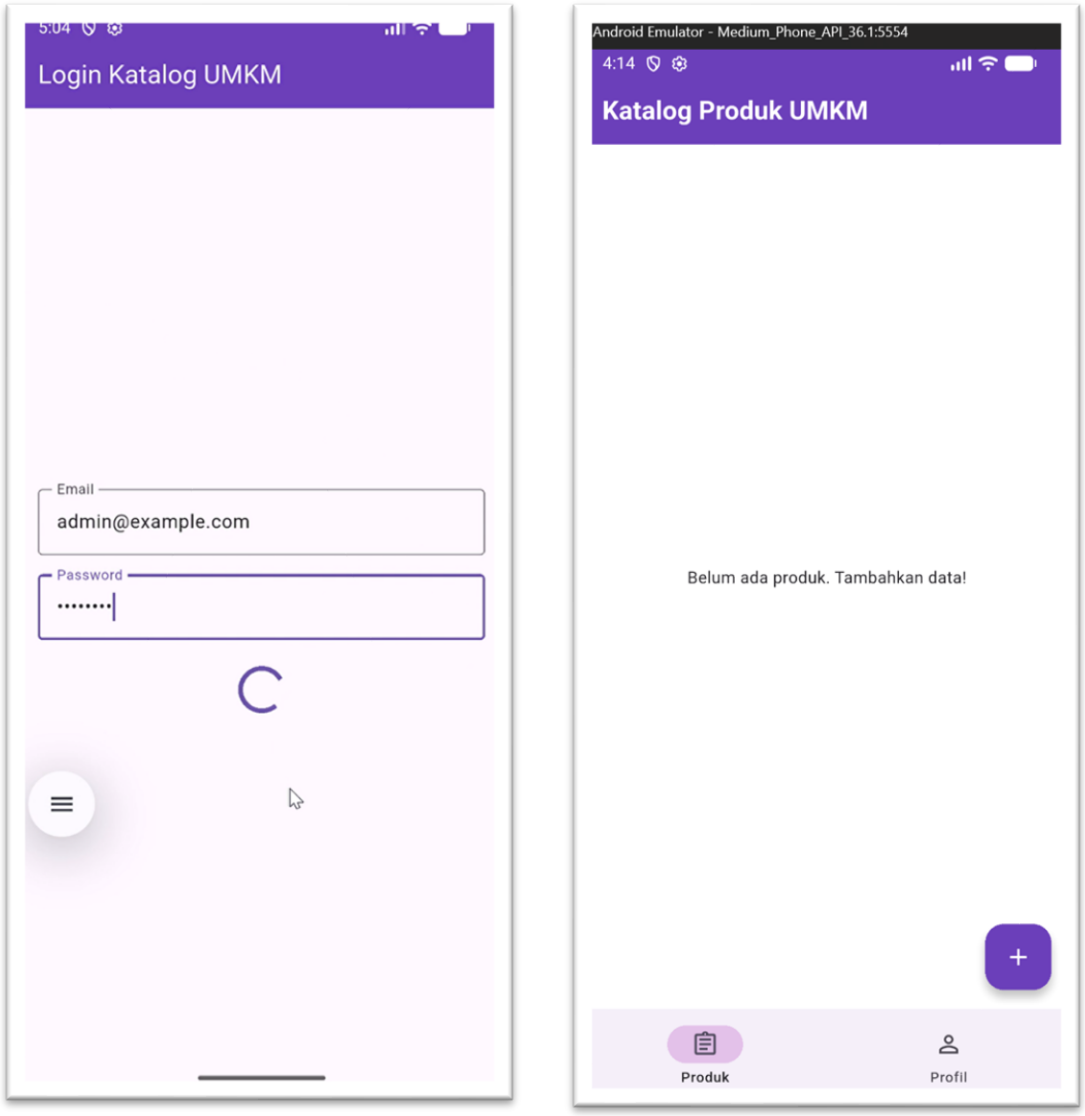
1. **Lapisan View (UI Layer):** Admin membuka halaman ProductFormView, mengisi form input teks, lalu menekan tombol "Tambah Produk". Fungsi FormState.validate() pada View akan memastikan seluruh inputan tidak kosong. Jika valid, View akan membuat objek Product baru dan memanggil fungsi di lapisan ViewModel: context.read<ProductViewModel>().addProduct(newProduct).
2. **Lapisan ViewModel (Business Logic Layer):** Kelas ProductViewModel menerima objek data produk tersebut dari View. ViewModel bertindak sebagai pengatur status (*State Manager*). Di dalam fungsi addProduct(), ViewModel akan memanggil fungsi eksekusi data yang ada di lapisan Repository: await _repository.insertProduct(product).
3. **Lapisan Repository (Data Access Layer):** Kelas ProductRepository bertindak sebagai gerbang data utama. Fungsi insertProduct() di dalam Repository akan mengambil koneksi database aktif dari DbHelper dan mengubah objek Product menjadi format Map biner (product.toMap()), kemudian mengeksekusi fungsi bawaan sqflite: db.insert('products', product.toMap()).
4. **Lapisan Penyimpanan (Persistence Layer):** Database mesin **SQLite** menerima perintah dan menyimpan data produk baru tersebut secara permanen ke dalam sektor memori lokal perangkat.
5. **Alur Sinkronisasi Balik (UI Reactive Update):** Setelah data sukses tersimpan di SQLite, fungsi addProduct() pada ViewModel segera memanggil fungsi asinkronus fetchProducts() untuk menarik data terbaru dari database, lalu mengeksekusi fungsi **notifyListeners()**. Perintah ini memicu sinyal reaktif ke lapisan View (ProductListView) untuk menggambar ulang antarmuka layar secara otomatis (*auto-refresh*) tanpa perlu memuat ulang seluruh sistem dari awal.

SCREENSHOT HASIL PENGUJIAN APLIKASI:



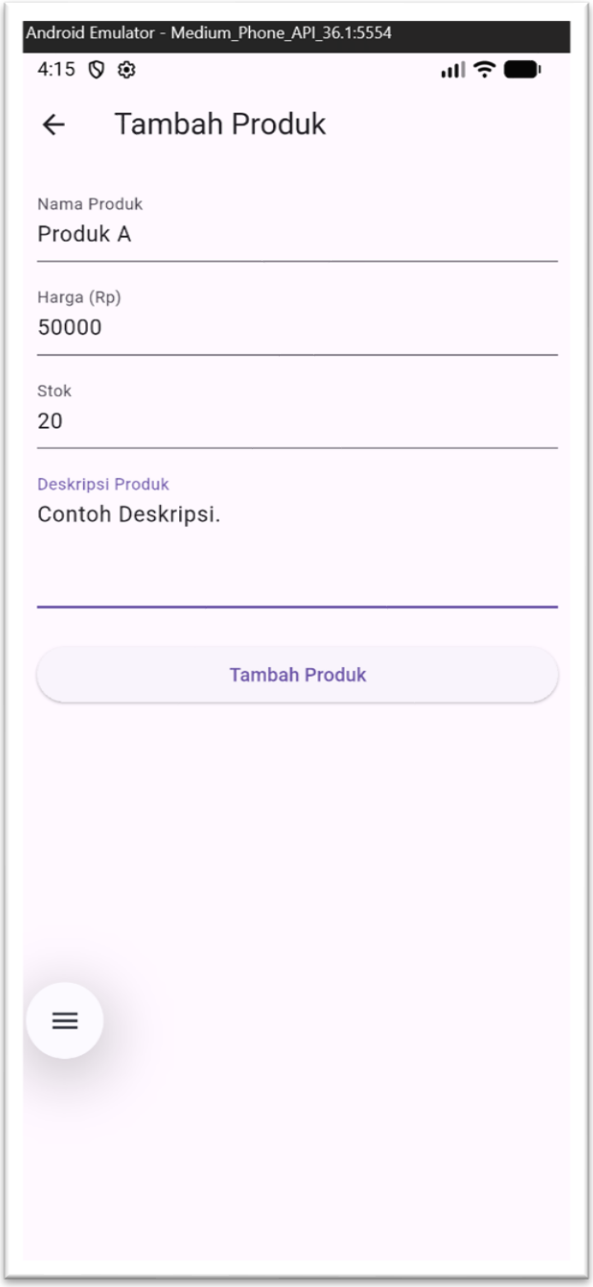
Screenshot Pesan Validasi Form Login Gagal

Keterangan: Tampilan pesan error merah "Email wajib diisi!" dan "Format email salah!" saat form login sengaja dikosongkan/diisi tidak sesuai standar.



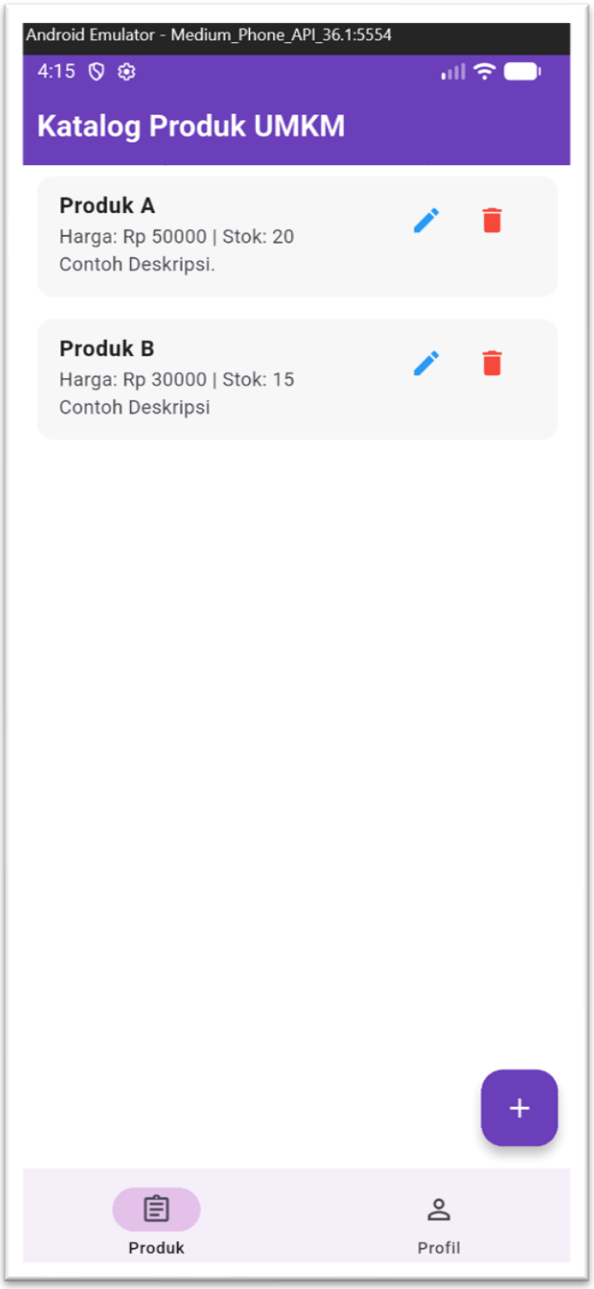
Screenshot Berhasil Masuk (Menu Utama - Produk)

Keterangan: Tampilan awal menu Produk yang bersih dengan AppBar ungu solid rata kiri dan tombol FloatingActionButton + setelah berhasil login menggunakan akun demo.



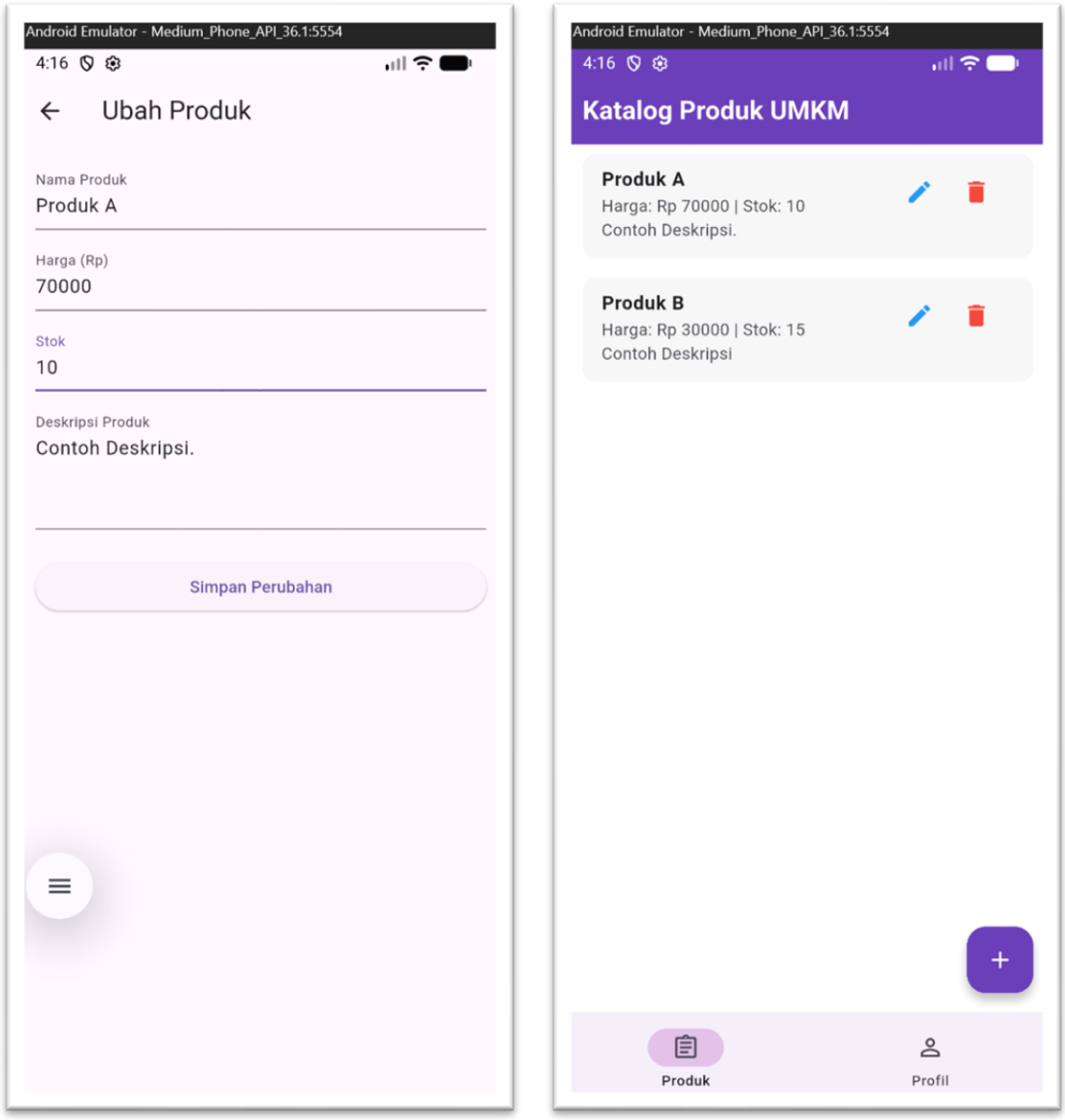
Screenshot Mengisi Form Tambah Produk

Keterangan: Dokumentasi pengisian form input data produk kerajinan UMKM yang memuat Nama, Harga, Stok, dan Deskripsi secara lengkap.



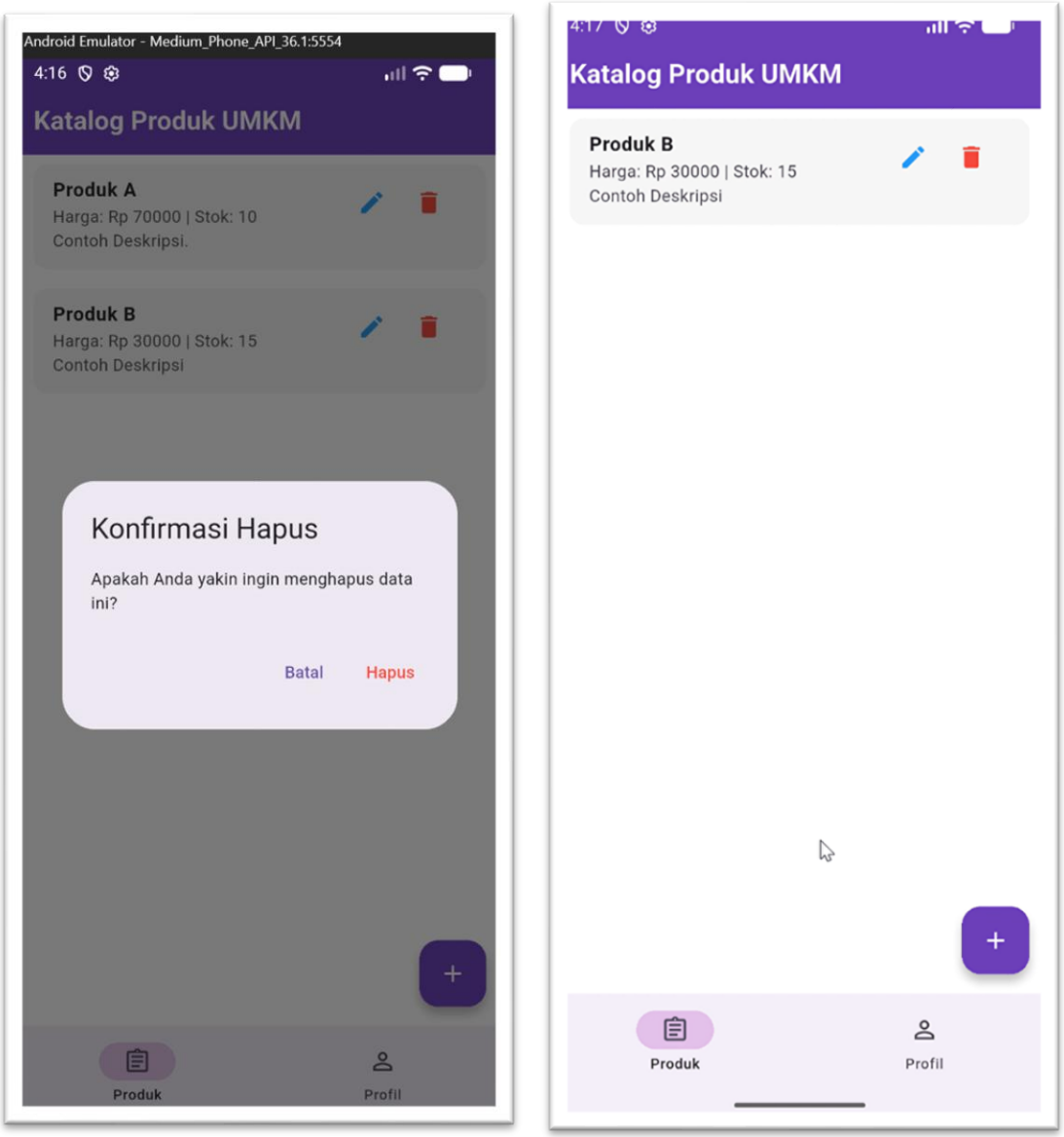
Screenshot Tampilan Daftar Katalog Produk

Keterangan: Tampilan halaman utama menu Produk yang menampilkan daftar barang kerajinan dalam bentuk komponen Card abu-abu tipis modern.



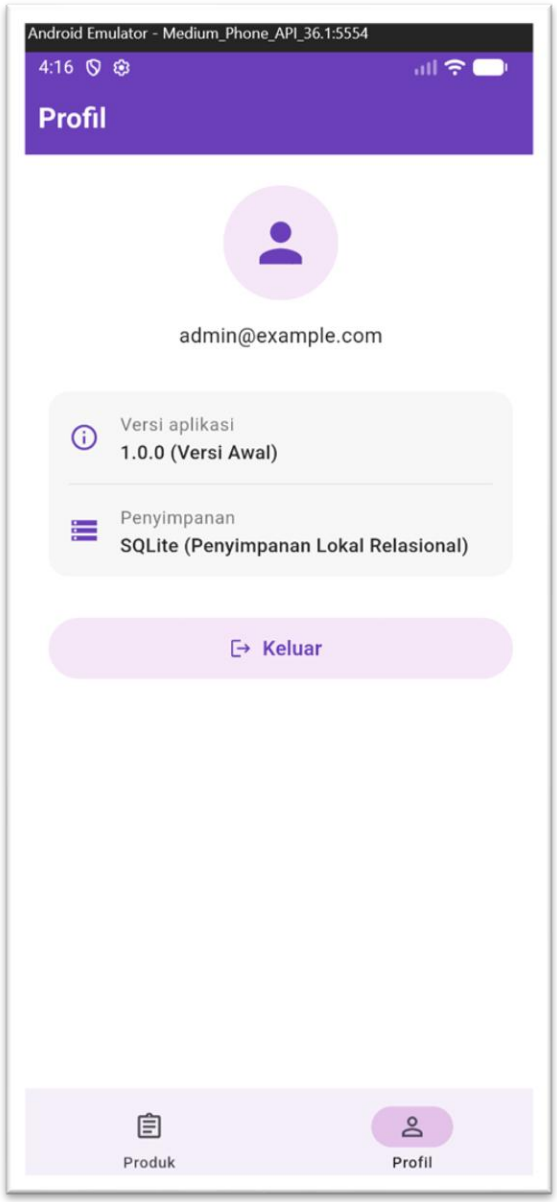
Screenshot Mengubah Data Produk

Keterangan: Mengubah data Produk A, dimana harga sebelumnya 50.000 dan stok nya 20, menjadi 70.000 dan 10.



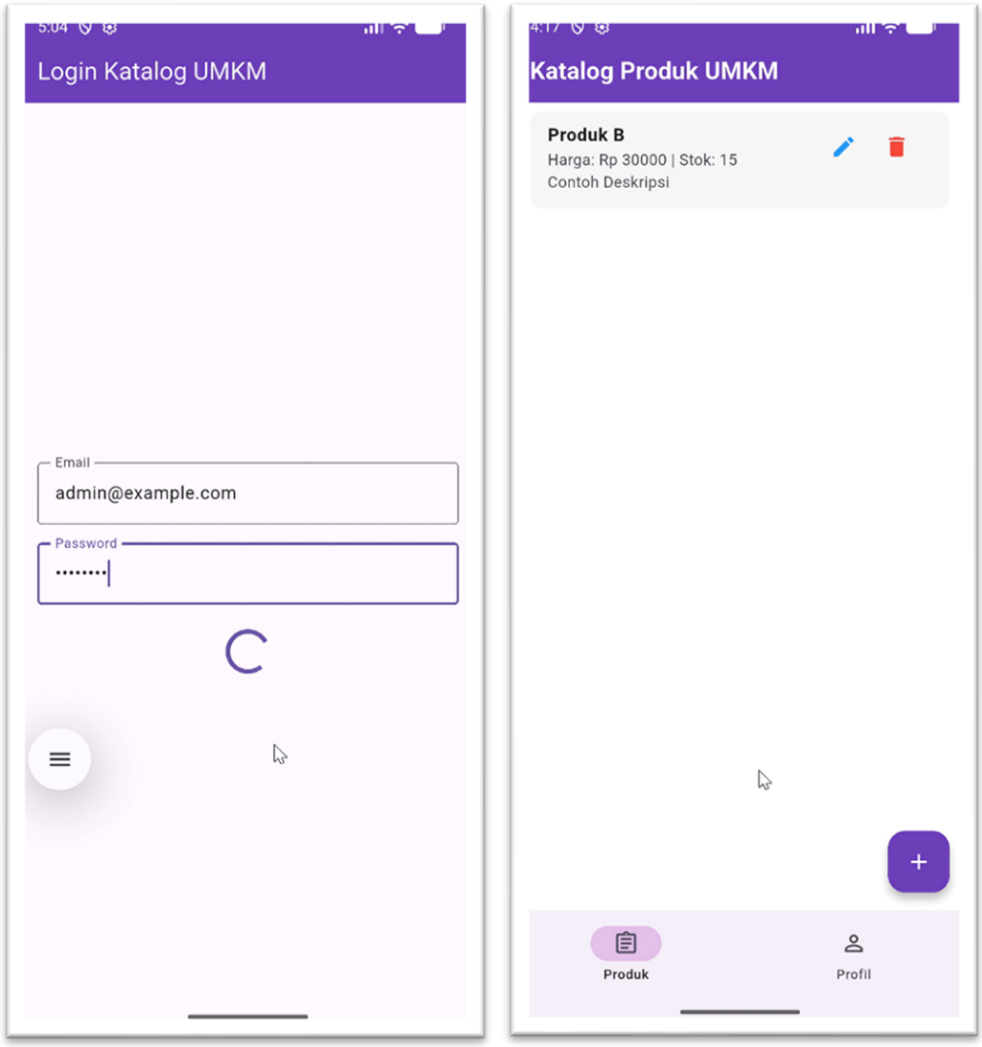
Screenshot Pop-up Jendela Dialog Konfirmasi Hapus Data

Keterangan: Menampilkan kotak dialog AlertDialog dengan opsi "Batal" dan "Hapus" yang muncul saat admin menekan ikon tempat sampah pada salah satu item katalog (Produk A).



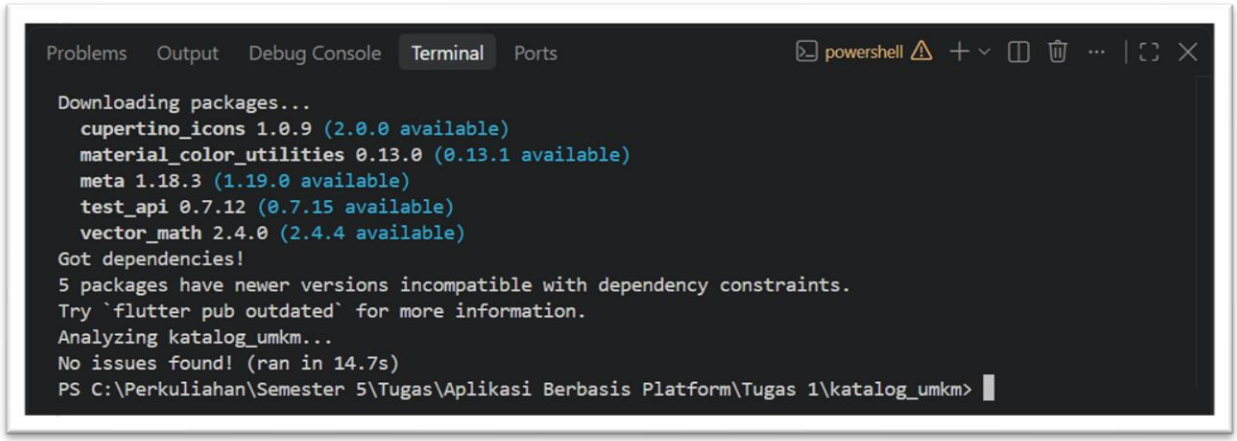
Screenshot Menu Profil

Keterangan: Dokumentasi halaman profil rata kiri dan menampilkan ringkasan info versi 1.0.0, jenis penyimpanan SQLite, serta tombol Keluar.



Screenshot Bukti Persistensi Data (Aplikasi Dibuka Kembali)

Keterangan: Bukti otentik bahwa data produk tetap tampil utuh di dalam sistem katalog tanpa ada kehilangan data setelah keluar/ logout, ataupun ditutup paksa dari background emulator dan dijalankan kembali.



Screenshot Bukti Hasil Flutter Analyze

Keterangan: Berdasarkan hasil pengujian menggunakan perintah flutter analyze, sistem menunjukkan bahwa tidak terdapat kesalahan atau masalah dalam kode program (*No Issues Found*). Hal ini mengindikasikan bahwa kode aplikasi telah memenuhi standar dan siap untuk digunakan.

KENDALA YANG DIHADAPI DAN CARA MENGATASI:

Selama proses pengerjaan dan perancangan Proyek Tugas 1 Aplikasi Berbasis Platform ini, terdapat beberapa kendala teknis yang dihadapi, di antaranya:

1. **Kendala 1: Kesalahan Analisis Struktur Jalur Impor (Target of URI Doesn't Exist)**
    - *Deskripsi Kendala:* Saat menjalankan perintah flutter analyze di terminal, sistem mengeluarkan banyak pesan error terkait kegagalan pendeteksian kelas Product dan ProductRepository di dalam file ViewModel akibat penulisan jalur impor data yang menggunakan metode relatif (../).
    - *Cara Mengatasi:* Kendala ini diselesaikan dengan merombak total seluruh arsitektur impor kode di bagian atas file menjadi bentuk impor paket absolut resmi proyek Flutter (contoh: import 'package:katalog_umkm/data/models/product_model.dart';), sehingga dependensi file dapat dilacak secara akurat oleh compiler.
2. **Kendala 2: Peringatan Pelanggaran Celah Asinkronus (Don't use BuildContexts across async gaps)**
    - *Deskripsi Kendala:* Muncul pesan peringatan (*info warning*) pada file login_view.dart karena aplikasi melakukan aksi perpindahan navigasi layar menggunakan komponen BuildContext tepat setelah proses asinkronus await simulasi loading login dijalankan. Hal ini dinilai berbahaya jika status widget telah hancur (*unmounted*) saat proses asinkronus selesai.
    - *Cara Mengatasi:* Masalah diatasi dengan menambahkan validasi pemeriksaan kondisi siklus hidup komponen widget menggunakan fungsi bawaan Flutter **if (!context.mounted) return;** tepat setelah baris kode asinkronus await

loginVM.login(...) dijalankan, sehingga operasi navigasi dipastikan aman dan lolos standarisasi analisis program.

3. **Kendala 3: Ikon Status Bar Sistem Hilang Saat AppBar Diwarnai**
    - *Deskripsi Kendala:* Ketika warna latar belakang komponen AppBar pada menu Produk dan Profil diubah menjadi warna ungu solid (Colors.deepPurple), ikon indikator bawaan sistem ponsel (seperti sinyal, logo Wi-Fi, jam, dan persentase baterai) di bagian paling atas layar mendadak tidak terlihat karena warnanya ikut menjadi gelap.
    - *Cara Mengatasi:* Mengonfigurasi properti systemOverlayStyle di dalam widget AppBar dengan memasang instruksi objek **SystemUiOverlayStyle(statusBarIconBrightness: Brightness.light)**. Instruksi ini memaksa sistem operasi ponsel untuk mengganti seluruh warna ikon status bar sistem menjadi warna putih cerah agar kontras di atas bar warna ungu aplikasi.

ALAT BANTU PENGERJAAN:

Seluruh kode program Flutter dan rancangan dokumentasi yang tertuang di dalam laporan ini dikembangkan oleh saya sendiri dengan memanfaatkan panduan arah dari alat bantu Gemini AI sebagai media asistensi pembelajaran dan juga materi-materi perkuliahan yang telah diberikan.